C++ Final Project: Create a Text Adventure Game

Overview

You will design and build your own **text-based adventure game** in C++. Your goal is to create an interactive game that uses **object oriented programming (classes)** to manage the game's components. This project is open-ended—you decide the storyline, characters, items, and challenges—but you must meet the core requirements.

Learning Objectives

- Practice designing and using **classes and objects**
- Reinforce control structures: conditionals, loops, and functions
- Work with user input and program output
- Develop modular, reusable code
- Practice software planning and design

Requirements

Your game must include:

1. **At least 6 classes** that represent different parts of the game (see suggestions below).
2. **A main game loop** that lets the player explore, make decisions, and progress.
3. **Inventory or status tracking** (e.g., player has a key, health is low, etc.).
4. **At least 10 rooms or scenes** with unique interactions.
5. **Meaningful user input**, such as commands ("go north", "take sword", "open door").
6. **Win/lose conditions** (e.g., escape the castle, find the treasure, survive the monster).
7. **Comments** and clear, organized code.

Suggested Classes

You are encouraged to design your own class structure, but here are some ideas to help you brainstorm:

- **`Room`**
  - Attributes: name, description, exits, items, is_locked
  - Methods: describe(), get_exit(), unlock(), add_item()

- **`Player`**
  - Attributes: name, current_location, inventory, health
  - Methods: move(), pick_up_item(), show_inventory(), take_damage()

- **`Item`**
  - Attributes: name, description, is_collectible, is_usable
  - Methods: use(), inspect()

- **`Player_class`**
  - Attributes: name, description, is_collectible, is_usable
  - Methods: use(), inspect()

- **`Enemy`**
  - Attributes: name, health, strength, location
  - Methods: attack(), take_damage()

- **`Game`**
  - Attributes: rooms, player, is_running
  - Methods: start(), parse_command(), run_game_loop()

Bonus Challenges (Choose 2)

- Add a map as a 2D array that reveals as the player explores.
- Use files to save/load game progress.
- Add randomness (e.g., dice rolls for combat or chance events).
- Include puzzles or locked doors that require solving riddles or finding keys.

Tips for Success

- Start by drawing a map or flowchart of your game.
- Plan your classes and how they interact before you write code.
- Break the project into small pieces and test as you go.
- Ask questions and collaborate with classmates when you're stuck!